ACT-R's Declarative Memory in CogLean: Laws, Two Explanations of Confident Errors, and the Power Law of Practice

Jeff Shrager

Draft, November 2026

Draft for Comment[1]

Abstract

CogLean is a library of cognitive theory in the Lean proof assistant, in which mechanisms are defined once, phenomena are stated independently of any mechanism, and theorems relating them are machine-checked; its theorems are explanations, proved about the same definitions that run as models. This paper collects what the library proves about one mechanism, the declarative memory of ACT-R: base-level activation, partial matching, softmax retrieval with a failure threshold, the latency rule, and priming. Three groups of results follow. First, laws: for every parameter setting in the rational regime used here, ACT-R's memory lacks determinacy, certainty, stability and order-freedom, and has recency and stored-only retrieval; consequences include graded confusion, interference with practiced items, and an order effect. Second, confident errors, a theory-neutral phenomenon that EPAM's discrimination nets produce exactly and that a theorem gives for every memory with three laws ACT-R lacks. ACT-R nonetheless approaches it: a graded version of the pattern needs no certainty, priming brings ACT-R's memory within any tolerance of it, and having just seen the familiar item is such priming, by the base level alone. The two explanations differ in an observation: after a familiar item is primed, ACT-R takes *other familiar* items for it too, which EPAM never does, and for the same reason primed ACT-R cannot satisfy a graded law of certainty. Third, the power law of practice, stated as a graded family of predicates. With decay 1, massed practice tested after a delay gives an exact power law, while spaced practice has every qualitative signature of a power law and is not one: it is logarithmic. With decay $d < 1$, using real analysis in a separate package, spaced practice is proved approximately a power law with exponent $1 - d$, which is Anderson's account, now checked. Whether ACT-R's memory produces a power law, and with what exponent, thus depends on the schedule and the decay, and the signature alone does not identify the mechanism.

1 Introduction

ACT-R [1, 2] is the most widely used cognitive architecture, and its declarative memory is one of its most thoroughly studied parts. Its equations are compact, its parameters are few, and its predictions are usually derived by running models. This paper does something complementary. It formalizes the memory's equations in CogLean [3, 4], a library of cognitive theory in the Lean proof assistant [5] organized on the model of Mathlib [6], and proves what follows from them: for every

[1]This draft has *not* been carefully checked by the author. It is likely to contain erroneous citations and details. Think of this as an over-starched blog post. I'm happy to receive constructive feedback at: `jshrager@gmail.com`

parameter setting in a stated regime, for whole classes of memories of which ACT-R's is or is not a member, and, at chosen settings, as exact values that serve as witnesses.

The companion papers introduce CogLean and its view that theorems are explanations: deductions of a phenomenon from mechanisms and stated assumptions [4]; compose theories into a model of multi-digit addition [7]; and take the library to concept formation [8]. ACT-R's memory appears in all of them, usually as a rival to EPAM's discrimination nets [9]. This paper gathers those results and adds two lines of work: a second explanation of confident errors, by priming, and the power law of practice.

Nothing here is a criticism of ACT-R or a new theory of memory. The point is that once a mechanism is stated precisely in a shared library, one can ask exactly which phenomena follow from it, under which conditions, and which observations would distinguish it from rivals. Several answers are not what informal reasoning suggests.

2 ACT-R's declarative memory, formalized

A chunk is a stored code, a vector of binary features, together with the times it was used (`CogLib/Memory/Activation.lean`). Memory is the current time and a list of chunks. Studying an item adds a use to its chunk, or creates one, and advances the clock.

Retrieval. A chunk i's activation for a probe x is its base level plus a partial-matching penalty for each feature on which chunk and probe differ, and retrieval chooses a chunk with probability proportional to $e^{A_i/s}$, or fails, with weight $e^{\tau/s}$ [1]. CogLean uses the regime in which every quantity is rational: decay $d = 1$ and temperature $s = 1$. Then

$$e^{A_i} \;=\; \Big(\sum_k \frac{1}{t - t_k}\Big)\,\mu^{m_i}, \qquad \mu = e^{-P}, \qquad \theta = e^{\tau},$$

where t_k are the chunk's uses, m_i the number of mismatching features and P the mismatch penalty. A retrieval event's probability is its share of the total weight (`Act.prob_retrieve`), and failure always has positive weight, so no retrieval is ever certain (`Act.retrieve_not_certain`).

Latency. ACT-R's retrieval time is Fe^{-A}. For a chunk retrieved by an exact match, at decay 1, that is F divided by the base-level sum (`Act.latency`).

Priming. Spreading activation from the context adds S_i to chunk i's activation, which multiplies its weight by e^{S_i}. CogLean represents this as a factor on a code's weight (`Act.retrieveIn`, with `Act.primeOnly y` ρ priming code y by ρ); with no priming, it is plain retrieval (`Act.retrieveIn_unprimed`), and a primed code's chance of retrieval is $\rho W_y/(\rho W_y + W_{\text{rest}})$ (`Act.prob_primed`).

What is left out. Other decays and temperatures need real exponentials; Section 5.3 uses real analysis, in a separate package, for learning curves only. Retrieval does not strengthen what it retrieves (ACT-R's retrieval as use), so memory changes only by study. There is no production system, no buffer, and no noise beyond the softmax choice.

3 Which laws ACT-R's memory has

CogLib states laws of retrieval theory-neutrally, against an interface any memory implements: study an item, and retrieve a distribution over stored items (`CogLib/Phenomena/RetrievalLaws.lean`). CogLean's theory matrix proves, for each memory family and law, whether the family has the law at every setting, never has it, or lacks it at some setting (`CogMatrix/`). For ACT-R's memory, with any positive failure weight and nonnegative mismatch factor, from an empty memory:

- it **never** has *certainty* (a just-studied item retrieves itself for certain), *stability* (studying one item leaves another's self-retrieval unchanged), *order-freedom* (swapping the last two study events leaves retrieval unchanged), or *determinacy* (every retrieval has a single outcome) (`Matrix.Activation.never_certainty`, `never_stability`, `never_orderFree`, `never_determinate`);
- it always has *recency*, with the later of two equally similar items retrieved twice as often, and *stored-only* retrieval (`Activation.recency`, `storedOnly`).

Each verdict is for every parameter setting in the regime, not one setting; a single-setting counterexample is the weaker kind of result, and the matrix keeps the two apart.

The laws have consequences across the library's tasks:

- **Graded confusion.** A probe can retrieve any stored item that partially matches it, weighted by similarity and recency. In learning a microwave, the label TIME COOK calls up SET TIME with probability 5/23, a kitchen timer with 10/23, and nothing with 8/23 (`DeviceLearning.act_graded`).
- **Interference.** Practiced items do not stay practiced. In the multi-digit model, studying $100 + 300$ raises the retrieval errors on the practiced $100 + 100$ from 225/782 to 3375/5842 (`MD.ExampleACTR.act_interference`); EPAM's net, by a theorem for every setting, is never disturbed.
- **An order effect.** In a counterbalanced study-test design written once against the memory interface, ACT-R's recency gives an order effect of 10/17, for every guessing bias, where order-free memories give 0 (`Exp.OrderEffect.act_report_any_bias`).

4 Confident errors: two explanations

4.1 The phenomenon, and the class ACT-R is not in

A stranger is confidently taken for someone familiar; a child who knows $100 + 100 = 200$ confidently answers $150 + 100$ with 200. CogLib states the pattern once, with no mechanism in it (`CogLib/Phenomena/ConfusionErrors.lean`). For a trial that recognizes a stimulus, retrieves an answer from the recognized item's associations, and otherwise backs up: (i) the familiar q is never answered wrongly from memory; (ii) every answer to the stranger x from memory is wrong; (iii) x backs up exactly as often as q. The general theorem `RR.confident_errors` gives the pattern when q calls up itself and x calls up q, *for certain*, and q's associations are sharp. `RR.confident_errors_of_laws` gives it for every memory with determinacy, certainty and stability [4]. EPAM's net has all three. ACT-R's memory has none, at any setting, so neither theorem applies.

Not covered is not the same as cannot. In naming familiar people (`Experiments/FaceNaming.lean`), with EPAM's net a stranger, Carl, is called Bob with

	$150 + 100$ answered 200 by retrieval	errors on practiced $100 + 100$
EPAM	$45/46 \approx .978$	0
ACT-R, unprimed	$225/529 \approx .425$	$225/782 \approx .288$
ACT-R, $100 + 100$ primed $\times 10$	$2250/2599 \approx .866$	
ACT-R, primed $\times 100$	$22500/23299 \approx .966$	

Table 1: The multi-digit setting [7]: EPAM's net, and ACT-R's memory unprimed and primed. Exact values by `decide`; witnesses, not fits.

probability 45/46 and says "don't know" as often as Bob does. With ACT-R's memory he is called Bob 225/391 of the time, but can also be called Ann (225/782), and says "don't know" more often than Bob (107/782 against 229/2254), so part (iii) fails (`act_values`, `act_more_dont_know`). In multi-digit addition, $150 + 100$ is answered 200 with probability 225/529, against EPAM's 45/46, and the exact pattern provably fails at that setting (`MD.Integrated.setting_actr_not_pattern`).

4.2 A graded pattern

Exactness is too strong a target for a memory whose retrieval is never certain. CogLib states a graded version: the pattern with each equality holding to within ε, and backup use on x at most r (`Phenomena.GradedConfidentErrors`); the exact pattern is the case $\varepsilon = 0$ (`ConfusionErrorPattern.graded`). The general theorem needs no certainty:

> `RR.confident_errors_graded`: if x calls up q, and q calls up itself, each with probability at least $1 - \delta$, and q's associations are sharp, the trial makes graded confident errors within 2δ, with backup use on x at most q's own exhaustion rate plus δ.

The proof bounds each part of the pattern by the probability mass that does not go to q, using two library facts about distributions: an expectation of a $[0, 1]$-valued quantity lies within the mass off an event of its value on the event (`Dist.expect_bounds`), and retrieval and backup factor through what is recognized.

4.3 Priming

ACT-R's memory meets the graded condition when the familiar item is primed. With q primed by ρ, the rest of the weight, for the probes x and q, shrinks relative to q's:

> `RR.act_priming_confident_errors`: for every $\delta > 0$ there is a priming factor ρ_0 such that, for every $\rho \geq \rho_0$, the primed trial makes graded confident errors within 2δ. Conditions: x partially matches q, q is stored, sharpness, well-formedness.

The explicit version gives the bound in terms of the weights (`RR.act_primed_confident_errors`). Since ACT-R's retrieval is never certain, the exact pattern is a limit approached, not a value reached. At the multi-digit setting, priming $100 + 100$ makes $150 + 100$ answered 200 by retrieval with probability $2250/2599 \approx .87$ tenfold and $22500/23299 \approx .97$ a hundredfold, against EPAM's $45/46 \approx .98$, and the two problems' backup use converges (`MD.ExampleACTR.act_primed_values`; Table 1).

4.4 What primes: having just seen the item

The priming factor need not be a free parameter. In ACT-R, a recent use raises a chunk's base level. Let `Act.rehearse s y j` add j uses of y one step before the test, without advancing the clock: y seen j times just before. Each such use adds 1 to the base-level sum of each of y's chunks, and the mismatch factor depends only on the code, so the effect on y's weight is the same factor for every probe:

> `Act.retrieve_rehearse`: retrieval after rehearsing y is retrieval with y primed by $\rho = 1 + kj/B$, where B is y's base level and k the number of its chunks; every other weight is unchanged.

So recency is priming, by a factor derived from the base level, and the priming theorem applies with nothing assumed about the context:

> `RR.act_recency_confident_errors`: for every $\delta > 0$ there is a number of uses of q just before the test beyond which the trial makes graded confident errors within 2δ.

In face naming, after Bob is seen ten times just before, Carl is called Bob with probability $275/299 \approx .92$, and his "don't know" $(1/26)$ comes close to Bob's $(629/20654)$ (`Exp.FaceNaming.recency_values`). Spreading activation from a context, the other source of priming, is not yet connected to this factor.

4.5 Telling the explanations apart

Confident errors now have two explanations in the library, from different mechanisms: discrimination, in which a net that has not yet tested the distinguishing feature sorts the stranger to the familiar item; and partial matching with priming, in which the familiar item's raised weight draws retrieval to it. They predict different things. EPAM's confusions follow the features a stranger shares with a stored item, and a familiar item is always recognized as itself. Primed ACT-R's follow the context, and priming draws retrieval away from *every* other item, familiar ones included. After Bob is seen ten times, Ann, a familiar face, is called Bob with probability $2475/3082 \approx .80$ (`recency_values`); EPAM's net never calls her Bob (`epam_ann_never_bob`). The priming explanation therefore predicts that a priming context raises misnaming of familiar people, as well as of strangers; the discrimination explanation does not.

The same fact says something about laws. CogLib states graded versions of the three laws: *determinacy within* δ (every probe puts at least $1 - \delta$ on one outcome), *certainty within* δ, and *stability within* η (studying another item lowers self-retrieval by at most η), together with proper retrieval. The exact laws are the case with tolerance 0 (`Laws.determinateWithin_zero` and the others), and the class theorem generalizes:

> `RR.confident_errors_of_graded_laws`: for any memory with the graded laws, after a study list xs without repeats, if x can be taken for a studied q with probability above the determinacy tolerance δ_d, the trial makes graded confident errors within $2(\delta_d + \delta_c + |xs|\,\eta)$.

Primed ACT-R cannot satisfy graded certainty for small tolerances, since priming q makes other studied items call up q. So its explanation of confident errors is not a law of the memory but a fact about the context, local to the primed item. The library makes that difference precise: one explanation follows from properties a memory has everywhere, the other from a condition that holds here and now.

5 The power law of practice

5.1 The phenomenon, unbundled

The power law of practice says that the time $T(n)$ to perform a task after n trials falls as an^{-b} [10]. As usually reported, it bundles claims of different strength. CogLib states them separately, as predicates on a curve $T : \mathbb{N} \to \mathbb{Q}$ over a range of trials (`CogLib/Phenomena/LearningCurve.lean`):

- `Improves`: $T(n+1) \le T(n)$;
- `DiminishingReturns`: each trial's gain is at most the last;
- `DiminishingRelativeReturns`: each trial's proportional gain is smaller than the last, $T(n+1)^2 < T(n)\,T(n+2)$;
- `ApproxScaleRatio k r` ε: within relative error ε of $T(kn) = r\,T(n)$;
- `ScaleRatio k r`: $T(kn) = r\,T(n)$, scale invariance, which is how an exact power law is stated with rationals ($r = k^{-b}$);
- `ShiftRatio s`: $T(n+1) = s\,T(n)$, an exponential curve.

The power curve a/n^m is scale-invariant with strictly diminishing relative returns (`power_scaleRatio`, `power_diminishingRelative`). An exponential curve has constant relative returns (`shiftRatio_not_diminishingRelative`), and no positive curve is both exponential and scale-invariant over a range (`shiftRatio_not_scaleRatio`): they are different phenomena. And an average of two exponential learners with different rates has strictly diminishing relative returns, and so is not exponential, though each learner is (`mix_diminishingRelative`, `mix_not_shiftRatio`). That is Estes's warning about group curves [11], central to the debate over whether individual learning is power or exponential [12], here as a theorem relating phenomena, with no mechanism at all.

The predicates are mathematical. Whether noisy data look like a power law, which might mean a power function fits well, fits better than an exponential, or merely that gains diminish, is a statistical proposition, which none of these settles.

5.2 Decay 1: one mechanism, two schedules

With decay 1, the base-level sum after n uses, tested at time t, is $\sum_k 1/(t - t_k)$, and latency is F divided by it (`CogLib/Process/Practice.lean`). Two schedules give curves of different shape:

- **Massed practice, tested after a delay** L: all n uses at once, so the sum is n/L and latency is FL/n, an exact power law with exponent 1 at every scale (`Act.massed_eq_power`, `Act.massed_scaleRatio`).
- **Spaced practice, tested after each trial**: one use per trial, so the sum is the harmonic number H_n and latency is F/H_n (`Act.spaced_eq`). It improves, with diminishing returns and diminishing relative returns (`spaced_improves`, `spaced_diminishing`, `spaced_diminishingRelative`): every qualitative signature of a power law.

But spaced practice is not a power law. Doubling practice adds at most 1 to H_n (`harmonic_double_le`), while H_n grows without bound ($H_{2^j} \ge j/2$, `harmonic_pow_two`), so the doubling ratio $T(2n)/T(n) = H_n/H_{2n}$ tends to 1. For any ratio $c < 1$, beyond any trial there is an

n with $T(2n) > cT(n)$ (`spaced_doubling`), so for any exponent above zero and any tolerance, the curve is not approximately scale-invariant over a long enough range (`spaced_not_powerLaw`). The curve is logarithmic. Over an experimentally typical range it is nonetheless close to one: over trials 1 to 8 it is within 7.5% of scale invariance with ratio 18/25 at doubling, a power law with exponent about 0.47 (`spaced_looks_like_powerLaw`, a witness). The doubling ratios at $n = 1, 2, 4, \ldots, 32$ are about $.67, .72, .77, .80, .83, .86$.

5.3 Decay below 1: Anderson's account, checked

ACT-R's usual decay is 0.5, and Anderson's account of the power law of practice rests on it [1, 13]: with decay d, the base-level sum after spaced practice is $\sum_{k \le n} k^{-d}$, which grows like $n^{1-d}/(1-d)$, so latency falls approximately as $n^{-(1-d)}$. That needs real powers, which CogLib, with exact rationals, does not have. A separate package, `ACTR/`, the only one in CogLean that uses Mathlib's real analysis, supplies them, and first connects them to CogLib's statements (`ACTR/PowerLaw.lean`):

- a rational curve is scale-invariant exactly when its real-valued copy is (`ACTR.scaleRatio_iff_cast`);
- the power law an^{-b} is scale-invariant with ratio k^{-b} for any real a and b (`rpow_scaleRatio`);
- conversely, scale invariance at doubling with ratio $r > 0$ is a power law on the doubling grid, $T(2^j) = T(1)\,(2^j)^{-b}$ with exponent $b = -\log_2 r$ (`powerLaw_of_scaleRatio`);
- CogLib's rational power curve is the real one, and its decay-1 spaced curve is the real curve at $d = 1$ (`power_cast`, `spaced_cast`).

Then, for every decay $0 \le d < 1$ (`ACTR/Decay.lean`):

- the sum lies between integrals of x^{-d}, $\frac{(1+n)^{1-d}-1}{1-d} \le \sum_{k \le n} k^{-d} \le 1 + \frac{(1+n)^{1-d}-1}{1-d}$ (`baseSpaced_bounds`), so divided by $(1+n)^{1-d}$ it tends to $1/(1-d)$ (`baseSpaced_div_tendsto`);
- the doubling ratio of latency tends to $2^{-(1-d)}$ (`spacedR_doubling_tendsto`);
- so spaced practice is approximately a power law with exponent $1 - d$: for every tolerance $\varepsilon > 0$, beyond some trial, doubling practice multiplies latency by $2^{-(1-d)}$ to within relative error ε (`spacedR_approx_powerLaw`).

The result is asymptotic. It says nothing about how many trials "beyond some trial" is for a given ε, which the integral bounds could make explicit.

5.4 What the practice results say

Three points follow. Whether ACT-R's memory produces a power law, and with what exponent, depends on the schedule and on the decay, not on the mechanism alone: massed practice gives exponent 1 at every decay considered here; spaced practice gives exponent $1 - d$ for $d < 1$ and no power law at $d = 1$. A curve can have every qualitative signature of a power law over a typical range without being one. And the signatures do not identify a mechanism: spaced base-level learning at $d = 1$ and an average of exponential learners share them. A claim that some mechanism "explains the power law of practice" should say which member of the family it means, under which schedule and parameters.

6 Discussion

What formalizing ACT-R's memory bought. Some results are verdicts for every parameter setting, where simulation gives one setting at a time: the four laws ACT-R never has, the two it always has. Some are explanations with their conditions explicit: graded confident errors, given priming; priming, given recent sightings; spaced practice as a power law, given $d < 1$. Some separate explanations: the Ann prediction, which tells primed partial matching from discrimination, and the failure of graded certainty, which says one explanation is a law of the memory and the other is not. And some correct informal impressions: a logarithmic curve with every signature of a power law, and a power law that depends on the schedule.

Limitations.

- Retrieval is at decay 1 and temperature 1, so that probabilities stay rational; real-valued decay is used only for learning curves, and real-valued temperature not at all.
- Retrieval does not strengthen what it retrieves. Full ACT-R, with retrieval as use, would give different values and possibly different theorems.
- Priming is a factor on a weight. Recency is derived as one source; spreading activation from a context is not yet connected to it.
- Learning curves concern latency of an exact-match retrieval; they do not include retrieval failure, production compilation, or the competition among chunks that partial matching brings.
- The decay result is asymptotic, without explicit bounds on when it takes hold.
- Nothing here is fitted to data. The values are witnesses.

Next.

- ACT-R's own model of the Siegler and Shrager addition data [14], which fits the answer distributions by partial matching with no backup strategy, criterion or learning, as a rival account of the same data, now that real exponentials are available.
- Retrieval as use, carried through the composed models.
- Spreading activation from a context as a second, derived source of priming.
- Which of the matrix's memory families have the graded laws, and at what tolerances.
- Other theories' learning curves on the same predicates, and statistical versions of the predicates for noisy data.

Conclusion. ACT-R's declarative memory, stated once in a shared library, yields verdicts for every parameter setting, explanations with their conditions explicit, and observations that tell its explanations from a rival's. Confident errors have two explanations, and the library says what would distinguish them and why only one is a law of the memory. The power law of practice turns out to be several phenomena, of which ACT-R's base level produces different ones under different schedules and decays. CogLean is not primarily about formalizing proofs. It is about formalizing explanations so that we can reason about them, and using Lean to keep those explanations rigorously aligned with the computational models they explain.

Code. All definitions and theorems are in the CogLean repository [3], `https://github.com/jeffshrager/CogLean`. Every theorem is fully proved, with no `sorry`, and depends only on Lean's standard axioms.

References

[1] John R. Anderson and Christian Lebiere. *The Atomic Components of Thought*. Erlbaum, Mahwah, NJ, 1998.

[2] John R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, New York, 2007.

[3] Jeff Shrager. CogLean: A library of cognitive theory in Lean 4. `https://github.com/jeffshrager/CogLean`, 2026.

[4] Jeff Shrager. CogLean: Towards a cumulative, machine-checked theory of cognition. Draft for comment, `papers/2026-coglean-intro/` in the CogLean repository, 2026.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[7] Jeff Shrager. Composing cognitive theories in CogLean: Multi-digit addition. Draft for comment, `papers/2026_multidigit/` in the CogLean repository, 2026.

[8] Jeff Shrager. Concept formation by view application: A second domain on shared foundations in CogLean. Draft for comment, `papers/2026-view-application/` in the CogLean repository, 2026.

[9] Edward A. Feigenbaum and Herbert A. Simon. A theory of the serial position effect. *British Journal of Psychology*, 53(3):307–320, 1962. doi: 10.1111/j.2044-8295.1962.tb00836.x.

[10] Allen Newell and Paul S. Rosenbloom. Mechanisms of skill acquisition and the law of practice. In John R. Anderson, editor, *Cognitive Skills and Their Acquisition*, pages 1–55. Erlbaum, Hillsdale, NJ, 1981.

[11] William K. Estes. The problem of inference from curves based on group data. *Psychological Bulletin*, 53(2):134–140, 1956.

[12] Andrew Heathcote, Scott Brown, and D. J. K. Mewhort. The power law repealed: The case for an exponential law of practice. *Psychonomic Bulletin & Review*, 7(2):185–207, 2000.

[13] John R. Anderson and Lael J. Schooler. Reflections of the environment in memory. *Psychological Science*, 2(6):396–408, 1991.

[14] Robert S. Siegler and Jeff Shrager. Strategy choices in addition and subtraction: How do children know what to do? In Catherine Sophian, editor, *Origins of Cognitive Skills*, pages 229–293. Erlbaum, Hillsdale, NJ, 1984.

A Index of results

Claim	Lean name (file)
Retrieval probability; never certain	`Act.prob_retrieve`, `Act.retrieve_not_certain` (`CogLib/Memory/Activation`)
Priming as a factor; primed share	`Act.retrieveIn`, `Act.retrieveIn_unprimed`, `Act.prob_primed`
Recency is priming	`Act.rehearse`, `Act.retrieve_rehearse`
Laws never had; laws always had	`Matrix.Activation.never_certainty`, `never_stability`, `never_orderFree`, `never_determinate`; `recency`, `storedOnly` (`CogMatrix`)
Graded retrieval; interference; order effect	`DeviceLearning.act_graded`; `MD.ExampleACTR.act_interference`; `Exp.OrderEffect.act_report_any_bias`
The pattern; graded pattern	`Phenomena.ConfusionErrorPattern`, `GradedConfidentErrors` (`CogLib/Phenomena/ConfusionErrors`)
Exact pattern fails for ACT-R (at a setting)	`MD.Integrated.setting_actr_not_pattern`, `Exp.FaceNaming.act_more_dont_know`
Graded confident errors, any memory	`RR.confident_errors_graded` (`CogLib/Process/RecognizeRetrieve`)
Priming gives graded confident errors	`RR.act_primed_confident_errors`, `RR.act_priming_confident_errors` (`CogLib/Process/PrimedRecognition`)
Recent sightings give them	`RR.act_recency_confident_errors`
Primed values	`MD.ExampleACTR.act_primed_values`, `Exp.FaceNaming.recency_values`
EPAM never misnames Ann	`Exp.FaceNaming.epam_ann_never_bob`
Graded laws; class theorem	`Laws.ProperRetrieval`, `DeterminateWithin`, `CertaintyWithin`, `StabilityWithin`; `RR.confident_errors_of_graded_laws`
Learning-curve predicates and relations	`Phenomena.Improves` … `ShiftRatio`; `shiftRatio_not_scaleRatio`, `mix_not_shiftRatio` (`CogLib/Phenomena/LearningCurve`)
Latency; massed and spaced practice, decay 1	`Act.latency`; `Act.massed_scaleRatio`; `Act.spaced_improves`, `spaced_diminishing`, `spaced_diminishingRelative`, `spaced_not_powerLaw`, `spaced_looks_like_powerLaw` (`CogLib/Process/Practice`)
Bridge to the reals	`ACTR.scaleRatio_iff_cast`, `rpow_scaleRatio`, `powerLaw_of_scaleRatio`, `power_cast`, `spaced_cast` (`ACTR/PowerLaw`)
Decay $d < 1$: approximately a power law, exponent $1 - d$	`ACTR.baseSpaced_bounds`, `spacedR_doubling_tendsto`, `spacedR_approx_powerLaw` (`ACTR/Decay`)